Python avancé

recto · itérateurs, fonctions, objets

`mot-clé type intégré # commentaire «à remplir»`

match : motifs avancés

1 · match · itérer

```
match msg:
    case {"op": "write", "adr": int(a), "data": d}:
        ecrire(a, d)                    # motif de dict
    case Point(x=0, y=y):               # motif de classe
        ...
    case [tete, *reste] if tete > 0:    # garde
        ...
    case int() | float() as v:          # alternative + nom
        ...
    case _:
        raise ValueError(msg)
```

Itérateurs

```
it = iter([1, 2, 3])  # appelle __iter__
next(it), next(it)    # 1, 2 (appelle __next__)
next(it, None)        # défaut, pas d'exception
class Compte:         # itérateur à la main
    def __init__(self, n): self.i, self.n = 0, n
    def __iter__(self): return self
    def __next__(self):
        if self.i >= self.n: raise StopIteration
        self.i += 1
        return self.i
```

Itérateur : `__iter__` + `__next__` ; ne se parcourt qu'une fois.

Générateurs

```
def lire_mots(chemin):          # génératrice
    with open(chemin) as fh:
        for ligne in fh:
            yield from ligne.split()  # délègue
mots = lire_mots("log.txt")  # rien n'est lu ici
for m in mots: ...           # lu à la demande
def fib():
    a, b = 0, 1
    while True:              # infini : sans souci
        yield a
        a, b = b, a + b
carres = (x * x for x in range(10**9))  # paresseux
```

`yield` suspend la fonction ; l'appel suivant reprend juste après. Mémoire constante.

itertools

`islice(it, 5)`	5 premiers d'un itérateur
`count(0, 2)`	0, 2, 4… à l'infini
`chain(l1, l2)`	enchaîner
`product(r, r)`	produit cartésien (boucles imbriquées)
`combinations(l, 2)`	paires, sans ordre ni répétition
`permutations(l)`	tous les ordres
`groupby(l, key)`	groupes consécutifs (trier avant)
`accumulate(l)`	sommes cumulées
`pairwise(l)`	(a, b), (b, c)… 3.10
`batched(l, 4)`	paquets de 4 3.12
`zip_longest(a, b)`	zip jusqu'au plus long

</> Fermetures (closures)

2 · fonctions

```
def compteur(debut=0):
    n = debut
    def inc(pas=1):
        nonlocal n          # modifier n du dessus
        n += pas
        return n
    return inc              # inc garde n en vie
c = compteur(); c(); c()  # 1, 2
fs = [lambda: i for i in range(3)]
[f() for f in fs]         # [2, 2, 2] : liaison
fs = [lambda i=i: i for i in range(3)]  # [0, 1, 2]
```

Une fonction est un objet : on la passe, la rend, la range dans un dict. Une variable libre est lue à l'appel, pas à la création.

</> Signatures

```
def f(a, b, /, c, *, d): ...  # a, b : par position seule
f(1, 2, 3, d=4); f(1, 2, c=3, d=4)  # d : par nom seul
```

</> Décorateurs

```
import functools, time
def timer(f):               # f -> nouvelle f
    @functools.wraps(f)  # garde nom, doc
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        res = f(*args, **kwargs)
        dt = time.perf_counter() - t0
        print(f"{f.__name__} : {dt:.3f} s")
        return res
    return wrapper
@timer                      # run = timer(run)
def run(n): ...
def retry(n=3):             # avec paramètres :
    def deco(f):            # un niveau de plus
        @functools.wraps(f)
        def wrapper(*a, **kw):
            for k in range(n):
                try: return f(*a, **kw)
                except OSError:
                    if k == n - 1: raise
        return wrapper
    return deco
@retry(n=5)
def telecharger(url): ...
```

Empilés, ils s'appliquent de bas en haut : `@a @b def f` donne `a(b(f))`.

functools

```
from functools import cache, lru_cache, partial, reduce
@cache                          # mémoïsation
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
@lru_cache(maxsize=256)         # les 256 derniers
def cout(cfg): ...              # args hachables
hex8 = partial("{:08x}".format)  # hex8(42) : 0000002a
```

Aussi : `@cached_property`, `@total_ordering`, `@singledispatch` (surcharge par type).

Méthodes spéciales

3 · objets

`__repr__ __str__`	`repr(o)` pour déboguer, `str(o)` et `print`
`__eq__ __hash__`	`==` ; redéfinir `__eq__` retire le hash
`__lt__ __le__...`	`<`, `sorted`, `max`
`__len__ __bool__`	`len(o)`, `if o:`
`__getitem__`	`o[i]`, tranches ; rend itérable
`__iter__`	`for x in o` ; `__contains__` : `x in o`
`__call__`	`o(x)` : objet appelable
`__add__ __radd__`	`o + x`, `x + o` ; `__iadd__` : `+=`
`__enter__ __exit__`	`with o:` (Context managers, au verso)
`__getattr__`	attribut introuvable

Classes : compléments

```
class Temp:
    __slots__ = ("_c",)  # pas de __dict__
    def __init__(self, c): self.c = c
    @property
    def c(self): return self._c
    @c.setter               # t.c = 20 valide
    def c(self, v):
        if v < -273.15: raise ValueError(v)
        self._c = v
    @classmethod            # autre constructeur
    def depuis_f(cls, f): return cls((f - 32) / 1.8)
from abc import ABC, abstractmethod
class Bus(ABC):             # classe abstraite
    @abstractmethod
    def lire(self, adr: int) -> int: ...
```

Héritage multiple

```
class Trace:  # mixin : un service
    def log(self, m): print(type(self).__name__, m)
class Uart(Trace, Periph): ...
Uart.__mro__  # ordre de recherche
```

`super()` appelle la classe suivante du MRO, pas forcément la mère écrite : l'appeler dans chaque `__init__`.

dataclasses et Enum

```
from dataclasses import dataclass, field, asdict, replace
@dataclass(frozen=True, order=True, slots=True)
class Run:
    freq_mhz: float
    nom: str = "base"
    tags: list[str] = field(default_factory=list)
    def __post_init__(self):  # validation
        if self.freq_mhz <= 0: raise ValueError
r = Run(100.0); asdict(r); replace(r, nom="opt")
from enum import Enum, IntFlag, auto
class Etat(Enum):
    IDLE = auto(); RUN = auto()
Etat.RUN.name, Etat["IDLE"], Etat(1)
class Perm(IntFlag):
    R = 4; W = 2; X = 1
Perm.R | Perm.W                 # combinable
```

Python avancé

verso · typage, concurrence, qualité

mot-clé type intégré # commentaire «à remplir»

Context managers

4 · with · typage · paquet

```python
class Chrono:
    def __enter__(self):
        self.t0 = time.perf_counter(); return self
    def __exit__(self, typ, val, tb):
        self.dt = time.perf_counter() - self.t0
        return False      # True : avale l'exc.
with Chrono() as c: travail()
print(c.dt)
from contextlib import contextmanager, suppress
@contextmanager
def dans(dossier):        # cd temporaire
    avant = os.getcwd(); os.chdir(dossier)
    try:
        yield             # corps du with
    finally:
        os.chdir(avant)   # même si exception
with dans("sim"), open("out.log", "w") as fh: ...
with suppress(FileNotFoundError): os.remove("x")
```

`ExitStack` : nombre variable de `with` ; `nullcontext()` : `with` qui ne fait rien.

Annotations de type

```python
from typing import Callable, Iterator, Literal, Protocol
def moy(xs: list[float]) -> float: ...
def trouver(k: str) -> int | None: ...  # Optional
cfg: dict[str, tuple[int, ...]] = {}
Mode = Literal["rtl", "gate"]
def appliquer(f: Callable[[int], str], x: int): ...
def premier[T](xs: list[T]) -> T:       # générique
    return xs[0]                        # 3.12
type Vecteur = list[float]              # alias 3.12
class Lisible(Protocol):                # duck typing
    def read(self, n: int = -1) -> bytes: ...
def charger(src: Lisible): ...          # a un .read()
def gen() -> Iterator[int]: yield 1     # génératrice
```

Rien n'est vérifié à l'exécution : lancer `mypy dossier` ou `pyright`. Avant 3.12 : `T = TypeVar("T")`.

Autres structures typées

```python
from typing import NamedTuple, TypedDict
class Pt(NamedTuple):  # tuple à champs nommés
    x: float; y: float
class Cfg(TypedDict):  # forme d'un dict (JSON)
    freq: float; seed: int
```

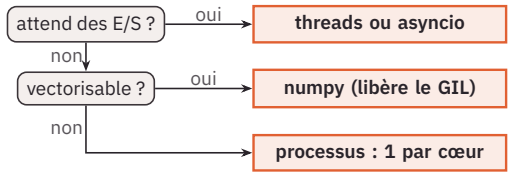
Paquet installable

```toml
# pyproject.toml
[project]
name = "outils-sei"
version = "0.1.0"
dependencies = ["numpy"]
[project.scripts]
bilan = "outils_sei.bilan:main"  # commande bilan
```

`pip install -e .` : installé en mode édition, `bilan` devient une commande.

Threads ou processus ?

5 · concurrence



GIL : un seul thread exécute du Python à la fois. Sans GIL : Python 3.13 `python3.13t` (expérimental).

concurrent.futures : N simulations

```python
import subprocess
from concurrent.futures import (ThreadPoolExecutor,
                                as_completed)
def simuler(seed: int) -> tuple[int, int]:
    r = subprocess.run(["vsim", "-c", "-do", "run.do",
                        "-sv_seed", str(seed)],
                       capture_output=True, text=True)
    return seed, r.returncode
with ThreadPoolExecutor(max_workers=8) as ex:
    futs = [ex.submit(simuler, s) for s in range(32)]
    for fut in as_completed(futs):  # ordre de fin
        seed, code = fut.result()
        print(seed, "OK" if code == 0 else "ÉCHEC")
```

Threads : ils attendent des processus externes. `ex.map(f, args)` : résultats dans l'ordre.

Calcul en parallèle

```python
from concurrent.futures import ProcessPoolExecutor
def cout(params): ...                   # pur calcul
if __name__ == "__main__":              # obligatoire
    with ProcessPoolExecutor() as ex:   # 1 par cœur
        res = list(ex.map(cout, grille, chunksize=16))
```

Fonctions et arguments sont copiés (pickle) vers chaque processus : pas de lambda, pas d'état partagé.

asyncio

```python
import asyncio
async def simuler(seed, sem):
    async with sem:                     # 8 à la fois max
        p = await asyncio.create_subprocess_exec(
            "vsim", "-c", "-do", "run.do",
            stdout=asyncio.subprocess.PIPE)
        out, _ = await p.communicate()
        return seed, p.returncode
async def main():
    sem = asyncio.Semaphore(8)
    res = await asyncio.gather(
        *(simuler(s, sem) for s in range(32)))
    async with asyncio.timeout(60):     # 3.11
        await asyncio.sleep(1)          # pas time.sleep
asyncio.run(main())                     # point d'entrée
```

`async def` crée une coroutine ; `await` rend la main à la boucle d'événements. Un seul thread : un appel bloquant fige tout (`asyncio.to_thread(f)` pour l'isoler).

Tests avec pytest

6 · qualité

```python
# tests/test_fifo.py : fichiers test_*.py
import pytest
from fifo import Fifo
def test_vide():
    assert Fifo(4).vide()      # assert simple
@pytest.fixture
def f():                       # préparation
    return Fifo(4)             # partagée
@pytest.mark.parametrize("n, plein",
                         [(3, False), (4, True)])
def test_plein(f, n, plein):  # f injectée
    for i in range(n): f.push(i)
    assert f.plein() == plein
def test_erreur(f):
    with pytest.raises(IndexError):
        f.pop()
assert 0.1 + 0.2 == pytest.approx(0.3)
```

`pytest -x -v -k plein` : arrêt au 1er échec, détaillé, filtre par nom. `tmp_path` : dossier temporaire fourni.

Débogage

```
breakpoint()              # pause ici (pdb)
python3 -m pdb script.py  # dès la 1re ligne
import pdb; pdb.pm()      # après un plantage
import traceback; traceback.print_exc()
```

`n s c`	ligne suivante, entrer dans l'appel, continuer
`p x`	afficher une expression ; pp joli
`l w q`	code autour, pile d'appels, quitter

Journal et mesures

```python
import logging
log = logging.getLogger(__name__)  # par module
logging.basicConfig(level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s")
log.info("run %d", n); log.exception("échec")
python3 -m timeit "sum(range(10**6))"
python3 -m cProfile -s cumtime script.py
```

`log.exception` ajoute la trace : à appeler dans un `except`.

Pièges avancés

Générateur épuisé	un 2e parcours est vide : refaire l'appel ou `list(g)`.
Attribut de classe	`tags = []` au niveau classe : une seule liste pour toutes les instances.
except: nu	attrape aussi `KeyboardInterrupt` : viser des exceptions précises.
__eq__ seul	l'objet n'est plus hachable (`set`, clé de `dict`).
Threads et CPU	aucun gain à cause du GIL : processus.
await oublié	`f()` seul crée la coroutine sans l'exécuter (« never awaited »).